

Comparative Analysis of Recurrent Neural Network Architectures for Time Series Weather Prediction

Yeşim Nur Tortop - 911967

June 11, 2024

Abstract

This report presents a comparative analysis of three recurrent neural network (RNN) architectures—Long Short-Term Memory (LSTM), traditional Recurrent Neural Networks (RNN), and Gated Recurrent Unit (GRU) models—applied to time series weather prediction. Using data from the ARPA Lombardia weather station in Lambrate (MI) from the early 2000s to April 2024, the study aims to forecast hourly temperature. Data preprocessing included normalization, handling missing values, and feature engineering. The models were evaluated for predictive accuracy, computational efficiency, and ability to capture temporal dependencies. This work highlights the potential of advanced RNN architectures in weather forecasting and provides a benchmark for future neural network applications.

1 Introduction

Weather forecasting is vital for various activities, including agriculture, disaster management, transportation, and daily planning. While traditional statistical and physics-based models are effective, they often struggle to capture the complex temporal dependencies present in weather data. Advanced machine-learning techniques, particularly Recurrent Neural Networks (RNNs), offer a promising alternative for improving the accuracy and reliability of weather forecasts.

This project explores the application of three prominent RNN architectures—Long Short-Term Memory (LSTM), traditional Recurrent Neural Networks (RNN), and Gated Recurrent Unit (GRU) models—to time series weather prediction. The dataset used in this study was obtained from the ARPA Lombardia weather station in Lambrate (MI)[5], encompassing records from the early 2000s to April 2024. Key weather parameters include temperature, wind speed, wind direction, global radiation, rain, and humidity.

The motivation for this project stems from the need to enhance weather forecasting capabilities by leveraging the strengths of advanced neural network architectures. LSTM, RNN, and GRU models are specifically designed to handle sequential data and can capture long-term dependencies, making them well-suited for time series prediction tasks. By comparing these models, we aim to identify their respective strengths and weaknesses, providing insights into their applicability for weather forecasting.

In this report, we detail the data preprocessing steps, including normalization, handling missing values, and feature engineering. We then describe the architecture and training process of each model, followed by an evaluation of their performance based on predictive accuracy, computational efficiency, and ability to capture temporal patterns in the data. The findings from this study highlight the potential of RNN architectures in enhancing weather prediction and set the stage for future research and development in this field.

2 Data Preprocessing

2.1 Data Exploration

The dataset used for this project consists of multiple CSV files, each containing different weather parameters collected from the ARPA Lombardia weather station in Lambrate (MI). The specific files and their corresponding parameters are:

Data	Date Range	Freq
Radiation	09/200304/2024	Hourly
Rain	06/200104/2024	10 minutes
Humidity	06/200104/2024	Hourly
Temperature	06/200104/2024	Hourly
Wind direction	06/200104/2024	Hourly
Wind speed	06/200104/2024	Hourly
PM10	04/200704/2024	Daily
PM25v	03/200704/2024	Daily

Table 1: Dataset Details

Upon initial examination, several key observations were made regarding the dataset:

- Date Range and Missing Data:** Files mostly cover the period from 06/2001 to 04/2024, except for the global radiation data and PM datas. A significant number of missing values were observed in the data prior to 2007. This inconsistency necessitates careful handling during preprocessing to ensure the integrity of the dataset.

- Missing Data Representation:** Missing values in the dataset are represented by the value ‘-999.0’. These need to be identified and treated appropriately, either through imputation or removal, to avoid skewing the model’s predictions.

- Data Sampling Intervals:** The rain data is recorded every 10 minutes, while other weather param-

eters are provided hourly. PM10 and PM2.5 data are given on a daily basis.

Overall, the data exploration phase highlights the need for extensive preprocessing to address missing values, normalize the data, and rectify any anomalies. This will ensure that the dataset is clean and ready for training the predictive models.

2.2 Data Cleaning

To ensure the dataset’s integrity and suitability for time series modeling, a comprehensive data cleaning process was conducted. This involved the following steps:

2.2.1 Consistency and Uniformity:

The dataset originally comprised nearly 200,000 rows. To maintain uniform temporal resolution, PM10 and PM25 data, which are recorded daily, were excluded from the analysis. Every CSV has a Id column that only includes same one number for every row. So this column is removed for every CSV file.

Rain data, initially recorded every 10 minutes, were aggregated to an hourly frequency by taking average of hourly data to match the other weather parameters. This adjustment ensured that all data have the same sampling rate, which is crucial for training an hourly time series model.

To simplify computation and reduce the dataset size, data prior to January 2016 and after January 2024 were removed. After this reduction, the dataset contained 72,312 rows, facilitating easier and more efficient computation.

2.2.2 Handling Missing and Wrong Data:

Missing values in the dataset are represented by ‘-999.0’. To address this, we identified the number of ‘-999.0’ values in each parameter:

- Wind Direction: 658
- Rain: 1,043
- Global Radiation: 471
- Temperature: 230
- Humidity: 152
- Wind Speed: 201

The total number of missing values across all parameters was less than 0.01 of the dataset. Therefore, these missing values were replaced with the mean of the five preceding and five succeeding values, ensuring the integrity of the data without significantly skewing the results.

Additionally, anomalies in the wind direction data, where values exceeded 360 degrees, were identified. Specifically, 961 instances of wind direction values greater than 360 degrees were replaced with the daily mean wind direction to prevent unrealistic values from impacting the model.

These data cleaning steps were essential to create a homogeneous and reliable dataset for training the time series models, facilitating accurate and efficient weather prediction.

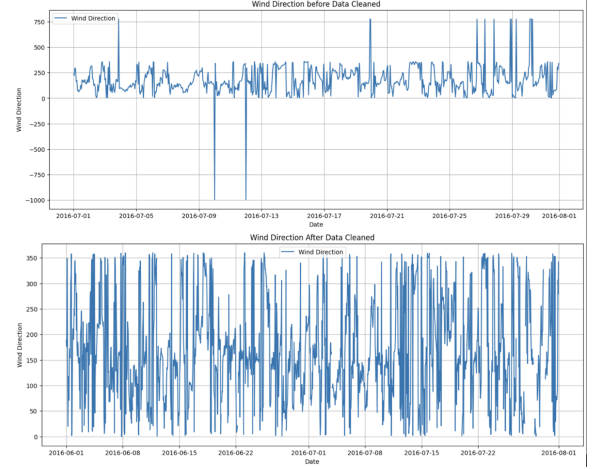


Figure 1: Wind Direction Data Before and After Cleaning

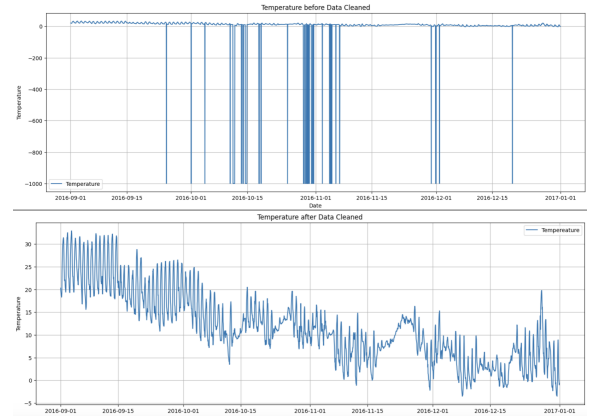


Figure 2: Temperature Data After Cleaning

2.2.3 Data Normalization

After cleaning the dataset, the next step involved normalizing the data using the MinMax scaler. Normalization is crucial for Recurrent Neural Network (RNN) architectures because it ensures that all input features are on a similar scale, typically between 0 and 1. This process enhances the efficiency of the learning algorithm and helps in faster convergence during training. By using the MinMax scaler, the data values are transformed based on the formula:

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

where X is the original value, $X_{\min}$ and $X_{\max}$ are the minimum and maximum values of X , respectively. This scaling is particularly beneficial for RNNs as it prevents features with larger scales from dominating the learning process and helps in maintaining numerical stability [8].

Following the normalization, the data was aggregated into monthly averages to facilitate easier visualization and interpretation. The figure below illustrates the monthly average values of the different weather parameters after normalization.

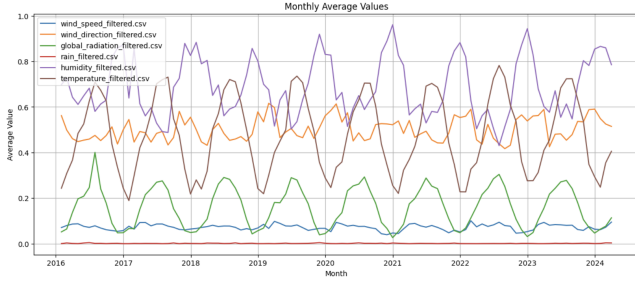


Figure 3: Monthly Average Values of Normalized Weather Parameters

The graph in Figure 3 provides several insights: **Trend Analysis:** The monthly averages reveal seasonal trends and variations in the weather parameters over the period from 2016 to 2024.

Correlation Identification: By observing the patterns, one can identify potential correlations between different weather parameters. For example, peaks and troughs in temperature and humidity often appear to be in sync.

Data Consistency: The visualization helps in verifying the consistency and reliability of the cleaned data, ensuring there are no significant gaps or anomalies remaining.

This visualization step is vital for understanding the underlying patterns in the dataset, which can inform and improve the predictive modeling process.

2.2.4 Data Manipulation for Wind Data

Following the normalization and aggregation, further data manipulation was performed on the wind direction and wind speed data to convert them into Cartesian coordinates (Wind.X and Wind.Y). This transformation allows for better handling of wind vector data in machine learning models by preserving the directional nature of the wind. The steps are detailed as follows:

1. **Conversion to Radians:** The wind direction data, originally measured in degrees, was converted to radians. This conversion is essential for trigonometric calculations and is represented by the formula:

$$\theta_{\text{radians}} = \theta_{\text{degrees}} \times \frac{\pi}{180}$$

where θ_{degrees} is the wind direction in degrees and θ_{radians} is the wind direction in radians.

2. **Calculation of Cartesian Components:** The wind speed and direction data were then converted into their respective Cartesian components (Wind.X and Wind.Y) using trigonometric functions. This is done to represent the wind as a vector:

$$\text{Wind.X} = \text{WindSpeed} \times \cos(\theta_{\text{radians}})$$

$$\text{Wind.Y} = \text{WindSpeed} \times \sin(\theta_{\text{radians}})$$

where WindSpeed is the magnitude of the wind vector and θ_{radians} is the wind direction in radians.

3. **Handling Zero Wind Speed:** In cases where the wind speed is zero, the Cartesian components (Wind.X and Wind.Y) are also set to zero. This step ensures that the transformation does not introduce erroneous values when there is no wind:

$$\text{if WindSpeed} = 0, \text{ then Wind.X} = 0 \text{ Wind.Y} = 0$$

4. **Dropping Unnecessary Columns:** After the transformation, the original wind direction (in degrees), wind speed, and intermediate radian conversion columns were dropped from the dataset to retain only the useful Cartesian components.

5. **Saving the Processed Data:** The final processed data, containing only the Wind.X and Wind.Y components, was saved to a new CSV file for subsequent use in modeling.

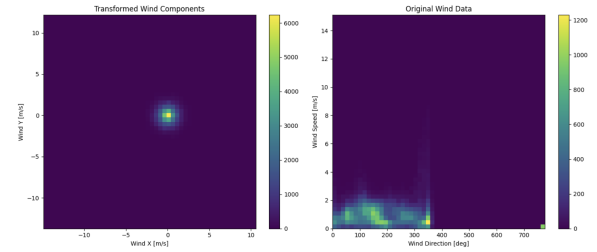


Figure 4: Wind Transformation

This transformation shown in Figure 4 is crucial as it converts polar wind data into a format that is more amenable to machine learning algorithms, which typically perform better with linear representations of features.

2.2.5 Preparing the Train, Test, and Validation Data

In time series forecasting, it is crucial to maintain the temporal order of the data during the train-test split. Unlike traditional datasets where the 'train-test-split' function from scikit-learn can be used to randomly split the data, time series data requires a sequential split to preserve the chronological order of observations.

To prepare the train, test, and validation datasets, the following steps were undertaken:

1. **Load the Dataset:** The dataset was loaded with dates parsed as the index, ensuring the preservation of the temporal sequence.

2. **Defining the Number of Samples:**

The total number of samples was determined, and based on the defined split ratios, the exact number of samples for each subset was calculated. Specifically, the training set included the first 70% of the data, the validation set the next 15%, and the test set the final 15%.

3. **Sequential Data Splitting:**

The data was split sequentially without shuffling to maintain the temporal order. This approach is essential to prevent data leakage and ensure that the model

is trained on past data and validated/tested on future data.

4. Sequence Creation:

Time series models like RNNs, LSTMs, and GRUs require input sequences for training. A sequence length (e.g., 24 hours) was defined to create input-output pairs. For each subset (training, validation, and test), sequences of the specified length were created to form the features (X) and the corresponding targets (y). This step involved iterating over the data to generate overlapping sequences that the models can learn from.

These steps ensured that the train, validation, and test datasets were prepared in a manner that preserves the temporal structure, facilitating effective training and evaluation of the time series models. By maintaining the chronological order, the model learns from past observations and generalizes well to future data, which is critical for accurate time series forecasting.

3 Methodology

Recurrent Neural Networks (RNNs) are a class of neural networks designed to recognize patterns in sequences of data, such as time series, text, or video frames. Unlike traditional feedforward networks, RNNs have connections that form directed cycles, enabling them to maintain a hidden state that captures information from previous time steps. This allows RNNs to effectively model temporal dependencies, making them well-suited for tasks where context and sequential order are important [6].

For weather prediction, RNN architectures are particularly advantageous due to the sequential nature of weather data. Current weather conditions are influenced by past states, making it essential to understand and model these temporal dependencies for accurate forecasting. RNNs excel in this domain by learning from historical data, capturing dynamic trends and temporal patterns crucial for reliable predictions. This makes RNNs a powerful tool for handling the complexities of time-dependent weather data.

RNNs process time series data by taking sequential inputs and recursively updating a hidden state at each time step. The hidden state acts as a memory that captures information about previous inputs. The process can be mathematically described as:

$$h_t = \sigma(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

where: h_t is the hidden state at time step t , x_t is the input at time step t , W_h and W_x are weight matrices, b is the bias term, σ is the activation function (e.g., tanh or ReLU)[3].

However, standard RNNs face challenges with long-term dependencies due to issues like vanishing and exploding gradients. To address these challenges, more advanced RNN architectures have been developed, such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) [1].

3.1 SimpleRNN

The Simple Recurrent Neural Network (SimpleRNN) is the most basic form of RNN, consisting of a single hidden state that is updated at each time step. SimpleRNNs are characterized by their simplicity and ease of implementation. However, they are often less effective at capturing long-term dependencies due to issues such as the vanishing gradient problem. The used SimpleRNN model structure given as follows:

Layer (type)	Output Shape	Param #
simple_rnn.2 (SimpleRNN)	(None, 24, 50)	2,850
dropout.2 (Dropout)	(None, 24, 50)	0
simple_rnn.3 (SimpleRNN)	(None, 50)	5,050
dropout.3 (Dropout)	(None, 50)	0
dense.1 (Dense)	(None, 6)	306

Table 2: Simple RNN Model Structure

The training process incorporated early stopping with the following parameters:

Monitor: 'val-loss' (validation loss) Patience: 3 epochs (training stops if validation loss does not improve for 3 consecutive epochs)

Restore Best Weights: True (restores the model weights from the epoch with the best validation loss)

Min Delta: 1e-3 (minimum change to qualify as an improvement)

3.2 Gated Recurrent Unit (GRU)

Gated Recurrent Units (GRUs) are a simplified variant of LSTMs designed to address the vanishing gradient problem. GRUs combine the forget and input gates into a single update gate, which reduces the complexity and number of parameters compared to LSTMs. This simplification makes GRUs faster to train while still being effective at capturing long-term dependencies. The GRU cell is defined by the following equations:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t] + b)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

where: z_t is the update gate, r_t is the reset gate [2].

The created GRU model details shown in following table:

The GRU model was trained using a phased approach with different batch sizes to optimize the training process. The following parameters and settings were used:

1. Batch Sizes: The initial training phase utilized a batch size of 32. In the subsequent training phase,

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 24, 50)	8,700
dropout (Dropout)	(None, 24, 50)	0
gru ₂ (GRU)	(None, 50)	15,300
dropout (Dropout)	(None, 50)	0
dense ₁ (Dense)	(None, 6)	306

Table 3: GRU Model Structure

the batch size was increased to 128 to enhance training efficiency and potentially improve model performance.

2. **Optimizer and Loss Function:** The Adam optimizer was employed for its efficient gradient-based optimization capabilities. The loss function used was Mean Squared Error (MSE), suitable for regression tasks such as weather prediction.

3. **Early Stopping:** Early stopping was implemented to prevent overfitting and ensure that the model did not train beyond the point of meaningful improvement. The parameters for early stopping were: Monitor: 'val-loss' (validation loss) Patience: 3 epochs (training stops if validation loss does not improve for 3 consecutive epochs) Restore Best Weights: True (restores the model weights from the epoch with the best validation loss) Min Delta: 1e-3 (minimum change to qualify as an improvement) Training was halted after 6 epochs due to early stopping, indicating that the model had reached optimal performance without further significant improvements in validation loss.

These training parameters and settings were chosen to ensure robust model performance and to efficiently handle the complexities of time series weather data.

3.3 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a type of RNN designed to better capture long-term dependencies. They achieve this through a complex cell structure that includes gates to control the flow of information. An LSTM cell has three main gates:

Forget Gate: Decides what information to discard from the cell state. **Input Gate:** Determines which new information to store in the cell state. **Output Gate:** Controls the output based on the cell state and the input.

These gates are defined by the following equations:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$h_t = o_t * \tanh(C_t)$$

where: - C_t is the cell state at time step t [4]. The LSTM models architecture shown in the following table.

Layer (type)	Output Shape	Param #
lstm ₄ (LSTM)	(None, 24, 50)	11,400
dropout ₁₆ (Dropout)	(None, 24, 50)	0
lstm ₅ (LSTM)	(None, 50)	20,200
dropout ₁₇ (Dropout)	(None, 50)	0
dense ₈ (Dense)	(None, 6)	306

Table 4: Model Architecture and Parameters

3.4 Bidirectional LSTM

Bidirectional Long Short-Term Memory (BiLSTM) networks are an extension of traditional LSTM networks. In a Bidirectional LSTM, two LSTM layers are employed: one processes the input sequence in the forward direction, while the other processes it in the backward direction. This structure allows the model to capture information from both past and future states for each time step, providing a more comprehensive understanding of the sequence context [7].

The architecture of a Bidirectional LSTM can be summarized as follows:

- **Forward Layer:** Processes the sequence from start to end.
- **Backward Layer:** Processes the sequence from end to start.
- **Concatenation:** The outputs from both layers are concatenated, combining information from both directions.

This approach enhances the model's ability to understand dependencies in the data, making it particularly effective for tasks where context from both past and future states is crucial.

Layer (type)	Output Shape	Param #
bidirectional	(None, 24, 128)	36,352
dropout ₄ (Dropout)	(None, 24, 128)	0
bidirectional ₁	(None, 24, 128)	98,816
dropout ₅ (Dropout)	(None, 24, 128)	0
lstm ₆ (LSTM)	(None, 32)	20,608
dropout ₆ (Dropout)	(None, 32)	0
dense ₂ (Dense)	(None, 32)	1,056
dense ₃ (Dense)	(None, 6)	198

Table 5: Model Architecture and Parameters

4 Results

4.1 Results for SimpleRNN

The Simple RNN model shows strong predictive power for Temperature and Humidity, with high R^2 values and low MAE and sMAPE scores. For Radiation, while

the R^2 is high, the sMAPE indicates some issues with relative error, likely due to variability in radiation measurements. The model struggles with predicting Precipitation and Wind components (Wind-X and Wind-Y), as indicated by negative R^2 values and high sMAPE scores.

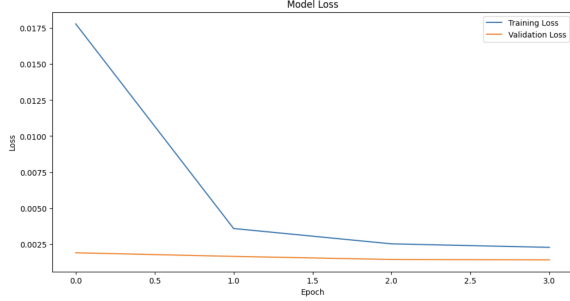


Figure 5: Simple RNN Model Loss

Due to the large batch size of 1534, the training loss decreased rapidly even within a single epoch as shown Figure 5. The early stopping mechanism triggered after 4 epochs, halting the training process to prevent overfitting.



Figure 6: Simple RNN Model Test Results

Upon analyzing the outputs shown in Figure 10, the model demonstrated better predictive performance for radiation, temperature, and humidity. However, it struggled to accurately predict rain and wind direction. The performance metrics of the model are summarized in the following table:

Metric	Wind_X	Wind_Y	Rad
MAE	0.0273	0.0275	0.0483
RMSE	0.0381	0.0379	0.0671
R^2	-0.0269	-0.0489	0.9191
sMAPE	4.6770	5.2219	126.5

Table 6: Performance Metrics for Wind_X, Wind_Y, and Rad

Metric	Rain	Hum	Temp
MAE	0.0031	0.0361	0.0207
RMSE	0.0135	0.0499	0.0270
R^2	-0.0182	0.9561	0.9796
sMAPE	198.8	6.0419	5.3891

Table 7: Performance Metrics for Prec, Hum, and Temp

4.2 Results for GRU Model

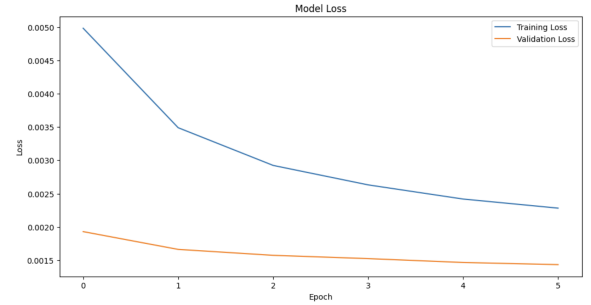


Figure 7: GRU Model Results

The GRU model was evaluated with batch sizes of 32 and 128, showing varying performance across different weather features. For Temperature and Humidity, both batch sizes yielded high R^2 values and low sMAPE scores, effectively capturing temporal dependencies. However, Radiation predictions, despite high R^2 , had significant relative errors (high MAPE and sMAPE) due to data variability and complexity. The larger batch size offered slight improvements but did not fully resolve these issues.

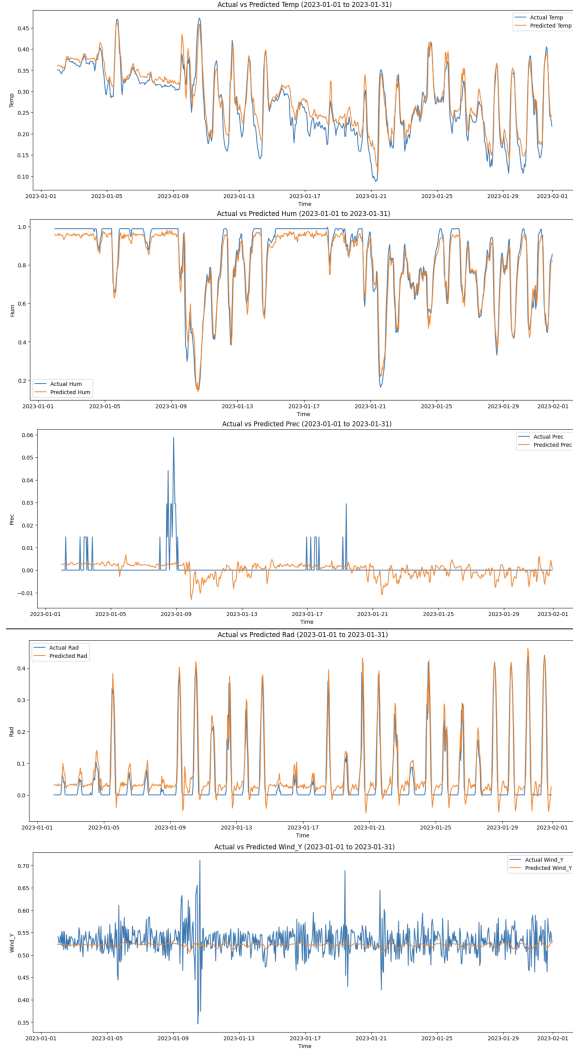


Figure 8: GRU Model Test Results

Predictions for Precipitation and Wind components (Wind-X and Wind-Y) struggled with negative R^2 values and high sMAPE scores, reflecting inherent data complexity. The larger batch size showed marginal improvements in these areas. In conclusion, the GRU model performs well for stable features like Temperature and Humidity but struggles with more variable features such as Precipitation and Wind components. Increasing the batch size from 32 to 128 improves predictions for Temperature and Humidity and shows moderate gains for Wind components.

The performance of the GRU model that trained with Batch size of 128 metrics for different features are summarized in the following tables:

Metric	Wind_X	Wind_Y	Rad
MAE	0.0273	0.0275	0.0483
RMSE	0.0381	0.0379	0.0671
R^2	-0.0269	-0.0489	0.9191
sMAPE	4.6770	5.2219	126.5

Table 8: GRU Performance Metrics for Wind_X, Wind_Y, and Rad

Metric	Rain	Hum	Temp
MAE	0.0031	0.0361	0.0207
RMSE	0.0135	0.0499	0.0270
R^2	-0.0182	0.9561	0.9796
sMAPE	198.8	6.0419	5.3891

Table 9: GRU Performance Metrics for Prec, Hum, and Temp

4.3 Results for LSTM

The LSTM model’s performance varies significantly across different weather features. It performs adequately for stable features like Temperature and Humidity, evidenced by high R^2 scores and moderate sMAPE values. This indicates that the model captures temporal dependencies effectively for these features, although there is potential for further optimization.

Metric	Wind_X	Wind_Y	Rad
MAE	0.0277	0.0285	0.0534
RMSE	0.0384	0.0390	0.0744
R^2	-0.0462	-0.1121	0.9005
sMAPE	4.7495	5.4098	128.2

Table 10: LSTM Performance Metrics for Wind_X, Wind_Y, and Rad (LSTM)

Metric	Prec	Hum	Temp
MAE	0.0063	0.0540	0.0274
RMSE	0.0142	0.0726	0.0354
R^2	-0.1269	0.9070	0.9649
sMAPE	196.5	9.1494	7.1561

Table 11: LSTM Performance Metrics for Prec, Hum, and Temp (LSTM)

However, the model struggles with Radiation despite its high R^2 value. The significant relative errors, reflected in the high MAE, RMSE, and sMAPE values, suggest that the model finds it challenging to handle the complexity and variability inherent in radiation measurements.

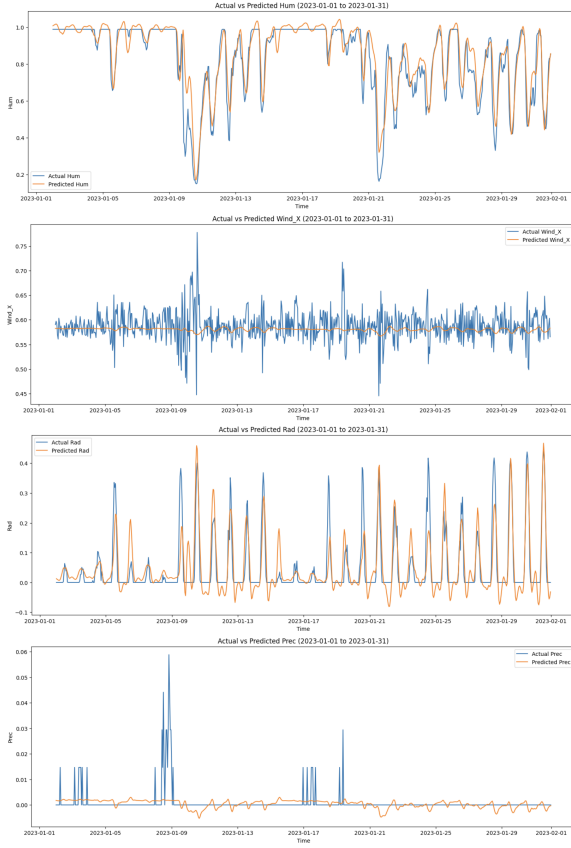


Figure 9: LSTM Model Test Results

In summary, while the LSTM model is relatively effective for predicting stable features such as Temperature and Humidity, it faces notable challenges with more variable features like Wind components, Radiation, and Precipitation.

4.4 Results for Bidirectional LSTM

The BigLSTM model demonstrates varied performance across different weather features. For stable features like Temperature and Humidity, the model performs reasonably well, as indicated by high R^2 scores and moderate sMAPE values. This suggests that the model effectively captures the temporal dependencies in these features, though there is still room for improvement.

For Radiation, despite achieving a high R^2 value, the model exhibits significant relative errors, as shown by high MAE, RMSE, and sMAPE values. This indicates that the model struggles with the variability and complexity of radiation measurements. The performance of the BigLSTM model across various weather features is summarized in Tables 12 and 13.

Metric	Wind_X	Wind_Y	Rad
MAE	0.0270	0.0270	0.0387
RMSE	0.0377	0.0375	0.0705
R^2	-0.0058	-0.0278	0.9106
sMAPE	4.6312	5.1187	136.2

Table 12: Performance Metrics for Wind.X, Wind.Y, and Rad

Metric	Prec	Hum	Temp
MAE	0.0032	0.0401	0.0274
RMSE	0.0135	0.0614	0.0354
R^2	-0.0212	0.9335	0.9649
sMAPE	198.8	6.9752	7.1561

Table 13: Performance Metrics for Prec, Hum, and Temp

The model faces significant challenges with more variable and sporadic features such as Wind components and Precipitation. The negative R^2 values and high sMAPE scores for these features highlight the model’s difficulty in capturing the inherent complexity and variability of the data.



Figure 10: Bidirectional LSTM Model Test Results

BigLSTM model performs well for stable features like Temperature and Humidity, it struggles with more variable features such as Wind components, Radiation, and Precipitation.

5 Conclusion

Evaluating SimpleRNN, GRU, LSTM, and BigLSTM models on weather prediction tasks revealed distinct strengths and limitations. The dataset included Temperature, Wind Speed, Wind Direction, Global Radiation, Rain, and Humidity, with preprocessing to handle missing values and normalize data.

SimpleRNN captured basic temporal patterns but

struggled with complex features, as shown by moderate MAE, RMSE, and R^2 values.

GRU models, tested with batch sizes of 32 and 128, excelled in predicting Temperature and Humidity but faced challenges with variable features like Wind components and Precipitation. Larger batch sizes improved performance for stable features and Wind components, but high errors persisted for complex features.

LSTM models performed well for Temperature and Humidity but had difficulty with Radiation, Wind components, and Precipitation, displaying significant relative errors despite some high R^2 values.

BigLSTM did not meet expectations, maintaining high R^2 for Radiation but showing high MAE, RMSE, and sMAPE values. It performed reasonably for Humidity but struggled significantly with Wind components and Precipitation, similar to other models. In summary, while recurrent neural network architectures like GRU and LSTM are effective for predicting stable weather features such as Temperature and Humidity, they encounter significant challenges with more variable features like Wind components, Radiation, and Precipitation. Wind data and rain data exhibit frequent and unexpected changes within hourly data, making them particularly difficult to predict. Even increasing the model complexity could not yield better results for these features. In contrast, Temperature, Humidity, and Radiation, which are more stable, are easier to predict even with simpler models like SimpleRNN. These findings suggest that further enhancements in preprocessing, feature engineering, or the adoption of more sophisticated or hybrid modeling approaches are necessary to improve predictive performance for highly variable features. The study underscores the importance of model selection and tuning in handling diverse temporal datasets, highlighting the need for continuous exploration of advanced methodologies in AI-based weather prediction.

References

- [1] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. In *IEEE Transactions on Neural Networks*, volume 5, pages 157–166. IEEE, 1994.
- [2] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [3] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [5] ARPA Lombardia. Weather data from arpa lombardia weather station. <https://www.arpalombardia.it/temi-ambientali/aria/form-richiesta-dati-stazioni-fisse/>, 2024. Accessed: 2024-06-10.
- [6] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.
- [7] Mike Schuster and Kuldip K Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.
- [8] Scikit-learn. Minmaxscaler. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>, 2021. Accessed: 2024-06-10.